

From 311 drone tiles to a 14 km inspection walk

AI vineyard inventory and inspection routes from one drone flight

Team **Solemtrix Hardware & Software**

Team lead: **TODO: Nume Prenume (team lead)**

Miroslav Maletici · Razvan Pervulescu · **TODO: alți membri**

GigaHack 2026 · Vineyard AI Field Challenge · Sireț3

27 September 2026

The problem: who checks every vine?

45 500 ha

of vineyards in the national vine registry...

66 400 ha

...of wine varieties in the official statistics (2024)

A 20 900 ha gap between the registry and the statistics.

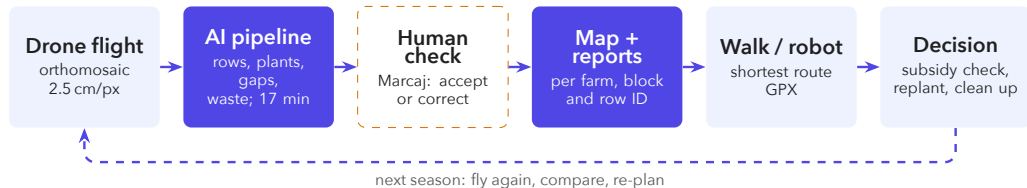
Planting subsidies reach up to 300 000 lei/ha, so every declared hectare needs evidence.

Roles and user stories

- 1 **State inspector (vine registry, subsidy control).** I want every block and row measured from one drone flight, so that I verify declared areas and plantings without walking every parcel.
- 2 **Farmer / vineyard owner.** I want a map of missing plants, gaps and waste per row, so that I replant and clean where it matters, before the harvest.
- 3 **Agronomist / field inspector.** I want the shortest closed walk through every problem spot, as GPX on my phone, so that one inspection takes 3.5 h instead of 17 h.
- 4 **Municipality (primăria).** I want farms, roads, cadastre parcels and dumped waste on one map, so that I plan land use and clean-ups.

Sources: agroexpert.md (registry vs statistics, 2024); telegraph.md (native-variety subsidy); maia.gov.md (vine registry). Full list: src/Web/docs/PREZENTARE_CERCETARE.md.

The business case: one flight, one map, one walk



Siret3, the challenge survey, end to end

81.5 ha
311 GeoTIFF tiles

27
farms

44
vineyard blocks

674
rows, 47.7 km

13 702
plants

1 226
inspection targets

Architecture: computer vision first, ML only where it pays

- 1 **Row axes: a 1-D signal.** Dominant angle, projection on the row normal, autocorrelation gives the spacing; $\text{SNR} \geq 3$ rejects orchards; Huber line fit; union-find links rows across tiles.
- 2 **Canopies: one polygon per plant.** Lab a* vegetation threshold, kept only inside the row corridor; connected components. A U-Net runs only where CV is blind (old vines in grass: 88 tiles, 13 s).
- 3 **Attributes: explicit rules.** $\text{Gap} \geq 5 \text{ m} \Rightarrow$ row *disrupted*; inter-row cover from the vegetation fraction; unclear image $\Rightarrow$ *unassessable*.
- 4 **Waste: a cascade.** Colour/shape rules $\rightarrow$ OpenCLIP probe $\rightarrow$ SAM 3 on the top crops $\rightarrow$ a human approves every box.
- 5 **Blocks, farms, route.** Cadastre parcels as a prior, road cuts; walking graph on inter-rows + passages; generalised TSP (OR-Tools, 2-opt), closed at START, $\leq 2\%$ outside.

$\approx$ 3 min

CV on all 311 tiles, laptop CPU
(measured)

0 labels

no training set needed; thresholds
set on the 2 example tiles

$\approx$ 18 h

SAM 3 on every tile instead
(extrapolated)

On the 2 labelled tiles CV scores 0.816 vs
0.641 for the U-Net alone (canopy).

Data pipeline: from GeoTIFF to route and reports



≈ 17 min

Tiles to deliverables on one laptop (M3 Pro), CPU only, 8 workers.

Cached by config hash

Re-runs reuse per-tile caches; a new config never reads stale ones.

One source of truth

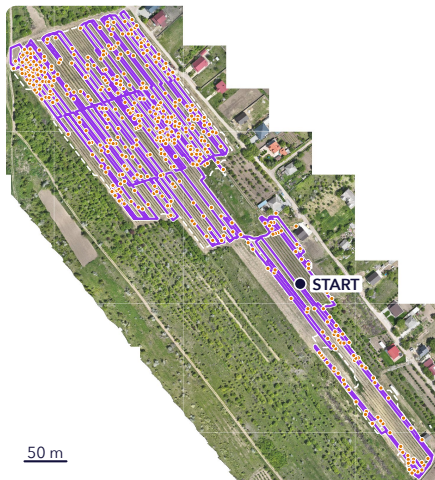
Every threshold lives in `default.yaml`; every run logs its config and timings.

Reproducible

Pinned `uv.lock`, 2 378 tests, CI, Docker with `--network none`. No paid API, no LLM at runtime.

Result: a route per farm, inter-rows only

SCORING · ROUTE 25%



— route ● targets - - - farm outline

Farm F09 (blocks V13 + V16), start picked on the map

9.7 km

closed walk, back to the start

455/455

targets of the farm visited

—65 %

vs a serpentine through every inter-row (27.5 km)

2 h 26 min

at 4 km/h; solved in 1.5 s in the browser

Whole survey: the official route.geojson

14.0 km

vs 68.2 km serpentine
(— 79 %)

147/147

reachable must-visit
targets

1.28 %

outside the allowed area
(limit 2 %); closed at START

On the organisers' 2 example tiles: row F1 1.00 (51/51), canopy score 0.876, inter-row IoU 0.96, counts score 0.973.

Demo: the web interface

Screencast

TODO: link YouTube screencast

Code and run instructions

```
github.com/miroslav43/  
Solemtrix_Hardware_Software__Vineyard_AI_Field_Challenge
```

Live demo from the team laptop (accepted by the rules):

```
cd src/Web/frontend && npm start  
→ http://localhost:3000/harta
```

What the screencast shows

- 1 **Map.** Orthophoto and every layer; click any object for its `vineyard_id` / `row_id`.
- 2 **Blocks and rows.** Counts, row lengths, canopy and inter-row areas (m², ha).
- 3 **Route.** Official route, START/FINISH, per-farm routes, GPX and GeoJSON.
- 4 **Farms, roads, cadastre.** 27 farms, public vs internal roads, live AGCC parcels. RO / EN / RU.

Pre-test point: 47.1225 N, 28.7095 E

Farm F09, block V13, inter-row V13-I020 (*mixed*), next to row V13-R020 (*disrupted*, 228.9 m, 70 plants). The route passes 1.8 m away: visited.

Impact: 13.5 h saved per inspection, 11-20 h to deploy

Economic effect, Siret3 (81.5 ha), walking at 4 km/h

3.5 h vs 17 h

one full inspection walk: 14.0 km instead of a 68.2 km serpentine

≈ 13.5 h

inspector time saved per survey (−79 %);
farm F09 alone: 2 h 26 instead of 6 h 52

≈ 0 lei

marginal compute: 17 min on a laptop
CPU, no paid API, no GPU, open weights

Deployment time to a ministry or a farm: ≈ 11-20 h to the first inspected region

One-time setup	hours
Server or laptop: Docker / uv sync, pnpm install, model weights from the release	1-2
Web app for inspectors: build, hosting, TLS, logins	2-3
Calibrate a new region: label 2 reference tiles, check thresholds	2-4
Train operators (inspector, agronomist)	1-2
Per new survey (≈ 80 ha)	
Ingest + AI pipeline (measured: 2.5 s + ≈ 17 min)	0.5
Human check in Marcaj	4-8
Final run: route, measurements, reports, publish	0.5

Compute times are measured (README §6). Human and set-up times are team estimates.

Scaling to industry

$\approx 4.7 \text{ s / ha}$

perception on one laptop, 8 workers (measured on 81.5 ha)

$\approx 130 \text{ h}$

all $\approx 100\,000$ ha of Moldovan vineyards on one laptop; ≈ 16 h on a 64-core server

Linear extrapolation of a measured run.
Tiles are independent, so it parallelises.

Features we want to offer the industry

- 1 **Ground robot, Street View for every row.** It drives our route (14 km instead of ≥ 41 km), side cameras image two rows per pass, on-board ML flags disease symptoms, each image tied to its `row_id`.
- 2 **Season-to-season change.** Fly again, compare: new gaps, replanting, abandoned plots, per row.
- 3 **Registry and subsidy integration.** Export per parcel to the vine registry (ONVV) and to AIPA field checks; cadastre (AGCC) parcels already on the map.
- 4 **Reports and routes per farm.** Per-farm route and GPX for phones and robots; CSV reports by block and row.
- 5 **On government infrastructure.** Offline Docker image, no external API; multi-user web app in RO / EN / RU. Other row crops (orchards, berries) reuse the same 1-D row model.

Summary

Efficient

17 min

tiles to deliverables, one laptop, CPU only

Accurate

1.00

row F1; canopy 0.876, inter-row IoU 0.96 (organisers' 2 example tiles)

Short routes

—79 %

whole survey vs serpentine: ≈ 13.5 h saved per walk

Deployable

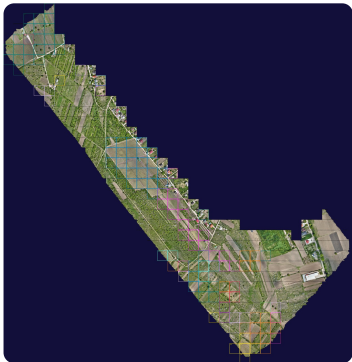
11-20 h

from zero to the first inspected region (estimate)

Thank you!

Questions?

The challenge: 81.5 ha, one plant at a time



Input. 311 GeoTIFF tiles at 2.5 cm/px (385 MB) over a 2 km strip of vineyards, orchards, meadows and village yards.

Output, 85 % scored on objective metrics:

- canopies, row axes, inter-rows, attributes and waste in Marcaj;
- measurements linked by `vineyard_id` / `row_id`;
- one closed walking route through every problem spot;
- a web map that shows all of it.

The real difficulty

Seeing one vine is easy. Seeing 13 702 the same way, telling them from young orchards and grass, then walking only to the ones that matter: that is the problem.

Why computer vision first, deep learning second

Canopy score on the organisers' 2 example tiles (the only labelled data)

CV: vegetation $\cap$ row corridor



CV $\cup$ U-Net



CV $\wedge$ U-Net



U-Net $\cap$ row corridor



U-Net alone



On this small sample the U-Net brings no gain, yet it costs training data and a GPU. So it runs only where CV is blind: old vines in grass, 88 tiles, 12.9 s.

Vines are periodic: structure is a stronger prior than pixels, and almost free to compute.

$\approx$ **3 min**

all 311 tiles on a laptop CPU (measured)

0 labels

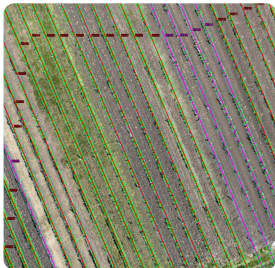
no training set; thresholds set on the 2 example tiles

$\approx$ **18 h**

SAM 3 on every tile (extrapolated)

Rows are a 1-D signal, not a 2-D image

SCORING · AXES 15%



Tile r006_c004: row axes and IDs.

674

rows, 47.7 km of
axes (whole survey)

44

blocks, one ID per
row across tiles

- 1 **Dominant angle, coarse to fine.** 2° , then 0.5° steps: about 100 evaluations instead of 360.
- 2 **Project onto the row normal.** A 1-D profile: autocorrelation gives the spacing (1.8–3.8 m), and an $\text{SNR} \geq 3$ rejects orchards.
- 3 **One robust line per peak.** Huber fit; union-find joins rows across tiles, one ID per row.

4 Find once, propagate

A tile with no confident row is searched again at its neighbours' lattice (angle $\pm 2^\circ$, spacing $\pm 15\%$).

Tested on the organisers' 2 example tiles

1.00

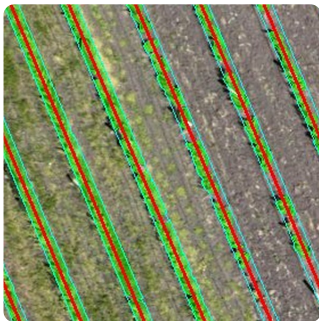
row F1 (51/51 rows)

1.00

grouping F1

0.05 %

row-length error



Green: canopy. Red: row axis. The grass on the left stays out.

13 702

plants, whole survey

149

disrupted rows

- 1 **One colour channel.** A threshold on Lab a*: no training, no GPU.
- 2 **Only inside the row corridor.** Grass between rows never becomes canopy.
- 3 **One polygon per plant.** Connected components: 13 702 plants, 1.54 ha of canopy.
- 4 **Attributes from geometry.** A gap ≥ 5 m marks a row *disrupted*; inter-row cover comes from the vegetation fraction.

Tested on the organisers' 2 example tiles

0.876

canopy score (official metric)

0.856

canopy IoU

0.905

canopy F1 (0.945 on tile r006_c004)

0.960

inter-row IoU; inter-row F1 1.00 (49/49)

1.00

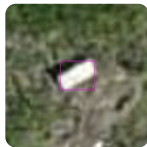
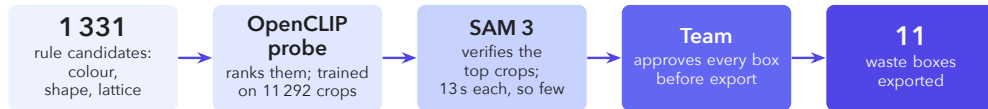
attributes: row structure 51/51, inter-row cover 49/49

585/650

reference plants matched

Waste: from cheap filters to big models

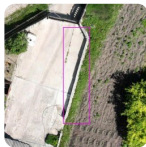
SCORING · WASTE 10%



exported



exported



rejected

Each stage is more expensive and sees fewer crops.

Precision over recall: nothing reaches Marcaj without an approval, so false boxes never cost points.

measurements.csv: survey → block → row

level / ID	rows	row length	canopy	plants
survey	674	47 737.5 m	15 427.5 m ²	13 702
block V01	115	9 955.4 m	3 262.9 m ²	2 346
block V13	77	13 551.2 m	3 471.8 m ²	5 525
block V41	83	4 089.9 m	1 469.4 m ²	831
... 44 blocks and 674 rows, each with its own line				

44

blocks

27

farms

9.32 ha

inter-rows

Web app (Next.js, RO / EN / RU)

- every layer: canopies, rows, inter-rows, waste, targets;
- official route and per-farm routes; GPX, GeoJSON;
- farms, roads, live AGCC cadastre; click any row for its numbers.

On the organisers' 2 example tiles

0.973

counts score (official metric)

51/51

rows counted; blocks 2/2

0.05 %

row-length error

1.4 %

canopy-area error (0.3 % pooled)

0.5 %

inter-row-area error

49/49

inter-rows matched

End to end, tiles → deliverables, current pipeline (RC10f), Apple M3 Pro laptop



Architecture

Typed stage contracts, one CLI command per stage, a torch-free core; ML is an optional extra.

Robustness

2 378 tests, CI on every push, an export validator: 311 tiles, 0 errors.

Scalability

Linear in area, per-tile workers, caches keyed by the config hash.

Reproducibility

Pinned uv.lock; Docker runs with `--network none`; free open weights.

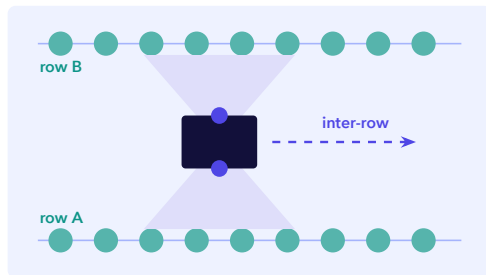
No paid API and no LLM at runtime. CPU only; the GPU is optional.

Next: Street View for every vine row

The drone map shows **where** something is wrong: a gap, a missing plant, waste. It cannot show **what** is wrong with a vine that is still standing.

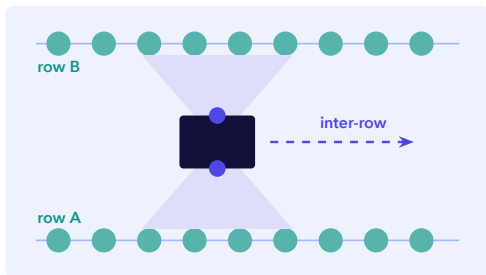
Our ground robot drives the vineyard like a Street View car:

- it follows the inter-rows of our route, never the rows;
- its cameras record geotagged images of both neighbouring rows;
- every image is linked to a `vineyard_id` / `row_id` from our map;
- the web map gets a street-level view of any row.



One inter-row pass, top view.

Disease detection on board



Left and right cameras: two rows per pass.

- 1 **Camera based.** Side-facing cameras see leaves and bunches, a scale the drone cannot reach.
- 2 **Machine learning on the robot.** A detector flags disease symptoms per image, on the device.
- 3 **Same IDs as the drone map.** Each detection carries its position, row and block, so it lands on the web map next to the gaps.
- 4 **Two rows per pass.** One camera per side: driving every second inter-row images every row, half the distance of a full sweep.

The drone finds where, the robot finds what



14.0 km

targeted route the robot drives

≥ 41.2 km

to cover every inter-row once

—66 %

distance, so battery and time,
per inspection

Backup: questions & answers

Validation · deep learning · scale · hard cases · route · reproducibility · time · data · users

How do we know the output is right?

Tested on the organisers' 2 example tiles (vineyard
eval-examples --gates, run rc10f)

metric	r006_c004	r021_c012	mean	gate
canopy score (official)	0.905	0.846	0.876	≥ 0.80 pass
canopy IoU	0.878	0.834	0.856	
canopy F1	0.945	0.864	0.905	
rows F1	1.00	1.00	1.00	≥ 0.95 per tile pass
inter-row IoU	0.963	0.957	0.960	≥ 0.95 pass
inter-row F1	1.00	1.00	1.00	
attributes	1.00	1.00	1.00	≥ 0.90 pass
row structure	26/26	25/25	51/51	
inter-row cover	25/25	24/24	49/49	
grouping F1	1.00	1.00	1.00	
counts score	0.973	0.974	0.973	

The example tiles contain no waste, so waste is checked by review, not by this table.

And over the whole survey

- a preview and a review priority for each of the 311 tiles;
- every row checked against its block's lattice (spacing, angle);
- rows follow the AGCC cadastral strips: median angle difference 0.3–0.6° (RC7);
- export validator: 311 tiles, 0 errors;
- route validator: closed at START, 1.28 % outside;
- manual correction in Marcaj before the final export.

Why not an end-to-end deep network?

Canopy ablation on the organisers' 2 example tiles (IoU / F1 / official score)

mask	tile r006_c004	tile r021_c012	mean
A vegetation $\cap$ corridor (CV)	0.834 / 0.887 / 0.855	0.762 / 0.799 / 0.777	0.816
B U-Net $\cap$ corridor	0.547 / 0.822 / 0.657	0.718 / 0.655 / 0.692	0.675
C vegetation $\wedge$ U-Net	0.547 / 0.822 / 0.657	0.743 / 0.779 / 0.758	0.707
D vegetation $\vee$ U-Net	0.830 / 0.887 / 0.853	0.730 / 0.638 / 0.693	0.773
E U-Net alone	0.539 / 0.806 / 0.646	0.657 / 0.607 / 0.637	0.641

Labels

Only 2 annotated tiles exist. A network would learn from our own CV pseudo-labels and inherit their errors.

Cost

U-Net: 0.30 s/tile, but needs training and a GPU. SAM 3: 13 s per crop.

Where it helps

Old vines in grass: the U-Net probability drives the cadastral completion on 88 tiles.

What happens at the scale of a country?

$\approx 4.7 \text{ s / ha}$

perception on this laptop, 8 workers (384 s for 81.5 ha, RC10f)

$\approx 130 \text{ h}$

for Moldova's $\approx 100\,000$ ha of vineyards, on one laptop

$\approx 16 \text{ h}$

on one 64-core server

Linear extrapolation of a measured run; not measured at that scale.

- **Tiles are independent:** perception parallelises across tiles with no shared state.
- **Global stages work on vectors:** row linking 3 s, blocks 1 s.
- **The route is bounded:** each OR-Tools solve has a time limit (30 s, 60 s for the final).
- **Re-runs are incremental:** caches are keyed by the config hash.

Shadows, grass, old vines, orchards?

Orchards, meadows

Rejected by periodicity: spectral $\text{SNR} < 3$, or a row wider than 1.2 m (p80).

Old vines in grass

A relaxed guided pass accepts rows up to 2.2 m wide; the cadastral completion searches on the U-Net probability.

Roads through vines

Rows are cut where an OpenStreetMap road crosses a vine-free stretch, so the road separates blocks.

Shadows

An inter-row more than 50 % in shadow is marked *unassessable*, not guessed.

Curved rows

Curved-row tracking follows the axis instead of forcing a straight line.

Left to people

41 tiles flagged for Marcaj. A missed row costs less than a false one.

How is the route built and checked?

- 1 **Domain.** Inter-rows $\cup$ authorised passages — forbidden zones — canopies. Inter-rows, never rows.
- 2 **Graph.** Inter-row centrelines plus passage skeletons, joined at the headlands.
- 3 **Targets.** Gaps, missing rows and plants, waste: 1 226, of which 246 must-visit. Each is a set of nodes it can be seen from.
- 4 **Solve.** Generalised TSP with OR-Tools (python-tsp fallback); several outside-share policies, the best valid one wins.
- 5 **Validate.** Closed at START, length, share outside the allowed area.

Baselines on the same graph

walk	length	saved
serpentine, every inter-row (est.)	68.2 km	79.4 %
every inter-row once (bound)	41.2 km	65.9 %
targets in ID order	36.0 km	61.0 %
ours	14.0 km	

211 min at 4 km/h. The serpentine estimate joins inter-rows with straight hops, which flatters the baseline.

The route reaches all 147 reachable must-visit targets. The other 99 are farther than the visit radius from any allowed path (23) or in parts no authorised passage connects to START (76).

Can someone else reproduce it?

```
# pinned environment
cd src/AI && uv sync --locked --all-extras
uv run vineyard doctor

# tiles -> Marcaj ZIPs
uv run vineyard all --workers 8

# Marcaj export -> route + measurements
uv run vineyard final <marcaj_export>.zip

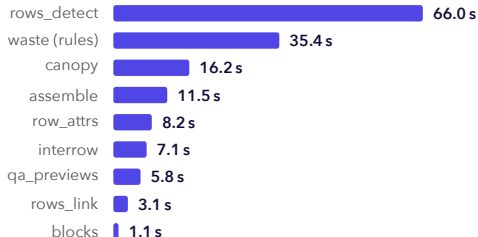
# or fully offline
docker run --network none ... vineyard all
```

- Python 3.12, exact versions in `uv.lock`; macOS arm64 and Linux.
- Weights are free and open (Hugging Face, Zenodo); nothing is downloaded at run time.
- Each run keeps its config, log and timings under `work/runs/<id>/`.
- CI: ruff and pytest (AI); lint, typecheck, build and Playwright end-to-end tests (web).

Where does the time go?

Stage wall times, 311 tiles, 8 workers, M3 Pro (run full-v2, rule-based waste)

Perception: 157 s



Post chain: 655 s



The route dominates by design: we spend the time budget on the solver, where it saves kilometres.

27 farms

blocks ≤ 10 m apart that no
public road separates

31.6 km

public roads, plus 56.3 km of field
tracks

- **AGCC cadastre** (public GeoServer): rows follow the cadastral strips, with a median angle difference of $0.3\text{--}0.6^\circ$.
- Parcel edges sit about 2.3 m (median) from ours, far above the 0.4 m row tolerance, so the cadastre is a **soft prior and a QA signal**, never a clipping mask.
- Parcels that already hold ≥ 3 rows are completed at their own angle and spacing.
- No owner data is stored, and nothing is taken from behind a login.

Who would use it?

45 500 ha

in the national vine registry...

66 400 ha

...of wine varieties according to the statistics bureau (April 2024)

- **Municipalities:** yearly land cadastre, abandoned land, illegal dumping, all on one map.
- **ONVV vine registry:** an independent, per-parcel check where registry and statistics disagree.
- **AIPA subsidies:** planting support up to 300 000 lei/ha for native varieties; targeted field checks with per-row evidence.
- **Why not satellites?** At 10 m a satellite sees the parcel; at 2.5 cm we see each plant, each gap and each piece of waste.

Sources: agroexpert.md (registry vs statistics, 2024); telegraph.md (native-variety subsidy); maia.gov.md (vine registry). Full list: src/Web/docs/PREZENTARE_CERCETARE.md.